

15. Задание: поиск по документации с RAG и LLM

Назначение

Нужно разработать утилиту, которая отвечает на вопросы по корпусу документации, используя подход **RAG** (Retrieval-Augmented Generation): сначала находит релевантные фрагменты документов семантическим поиском, затем передаёт их в **LLM** и формирует ответ со ссылками на источники.

Главное требование: ответ должен опираться **только на найденную документацию** и сопровождаться ссылками на конкретные документы/фрагменты. Если релевантной информации нет — утилита честно сообщает, что ответа в документации не нашлось, а не выдумывает его.

Основной результат

На выходе должен получиться проект docsearch — утилита на **Go 1.20**, которая:

1. индексирует корпус документации (Markdown/текст);
2. разбивает документы на фрагменты (чанки);
3. вычисляет эмбединги фрагментов и сохраняет их в локальном векторном хранилище;
4. по запросу находит top-k релевантных фрагментов через косинусную близость;
5. собирает контекст и формирует ответ через LLM со ссылками на источники;
6. обрабатывает случай «ответа нет в документации»;
7. конфигурируется через **YAML-файл**, секреты берёт из переменных окружения;
8. содержит тесты, включая offline-режим без обращения к внешним моделям.

Как устроен RAG-пайплайн

Этап	Назначение
Indexing	Парсинг документов, чанкование, эмбединги, запись в векторное хранилище
Retrieval	Эмбединг запроса, поиск top-k ближайших фрагментов по косинусной близости
Generation	Сборка промпта из найденного контекста и запрос к LLM с требованием цитировать источники

Формат входных данных

Вход — каталог с документами, заданный в конфигурации. Минимально поддерживаются форматы md и txt. Каждый документ нормализуется (убираются лишние пробелы, разметка приводится к тексту) и разбивается на чанки с перекрытием.

Для оценки качества (eval) используется размеченный набор в JSONL — пары «вопрос ожидаемые документы/ответ»:

```
{"query": "Linux?", "expected_docs": ["install.md", "service.md"]}
```

Формат конфигурации (YAML)

Ключи доступа к внешним API берутся из переменных окружения. Поддерживаются как локальные, так и удалённые модели; поля base_url/model задаются в конфигурации.

```
corpus:
  path: "./docs"
  formats: ["md", "txt"]          # md | txt | html | pdf

chunking:
  max_tokens: 512
  overlap_tokens: 64

embeddings:
  provider: "openai"              # openai | local
  model: "text-embedding-3-small"
  base_url: "https://api.openai.com/v1"
  # API-      EMBEDDINGS_API_KEY

vector_store:
  backend: "sqlite"              # sqlite | file
  path: "./index/vectors.db"

retrieval:
  top_k: 5
  min_score: 0.2                 # ; - « »

llm:
  provider: "openai"             # openai | mock
  model: "gpt-4o-mini"
  base_url: "https://api.openai.com/v1"
  # API-      LLM_API_KEY
  temperature: 0.1
  max_tokens: 1024

logging:
  level: "info"
```

Поля конфигурации:

Поле	Тип	Обязательное	Назначение
corpus.path	string	да	Каталог с документами
corpus.formats[]	array	да	Поддерживаемые форматы документов
chunking.max_tokens	integer	да	Максимальный размер чанка
chunking.overlap_tokens	integer	нет	Перекрытие между соседними чанками
embeddings.provider	string	да	Источник эмбедингов: openai или local
vector_store.backend	string	да	Хранилище векторов: sqlite или file
retrieval.top_k	integer	да	Сколько фрагментов извлекать
retrieval.min_score	number	нет	Порог релевантности
llm.provider	string	да	openai или mock (для тестов)
llm.model	string	да	Модель генерации

Переменные окружения:

Переменная	Назначение
EMBEDDINGS_API_KEY	Ключ доступа к API эмбедингов
LLM_API_KEY	Ключ доступа к API LLM

Выходной формат результата

Ответ на запрос в JSON:

```
{
  "query": "Linux?",
  "answer": "unit systemd: ... install.md.",
  "sources": [
    {"doc_id": "install.md", "chunk_id": 12, "score": 0.83, "snippet": "..."},
    {"doc_id": "service.md", "chunk_id": 3, "score": 0.71, "snippet": "..."}
  ],
  "model": "gpt-4o-mini",
  "tokens_used": 1840,
  "duration_ms": 2300
}
```

Поле	Назначение
query	Исходный запрос
answer	Ответ, сформированный LLM по найденному контексту
sources	Использованные фрагменты с документом, чанком и оценкой близости
tokens_used	Сколько токенов потрачено на запрос
duration_ms	Время обработки запроса

Если релевантных фрагментов нет (все ниже `min_score`), поле `answer` должно явно сообщать, что ответа в документации не найдено, а `sources` быть пустым.

CLI

Обязательные команды:

```
docsearch index --config config.yml #
docsearch ask --config config.yml --query "" # stdout
docsearch ask --config config.yml --query "" --out answer.json
```

Дополнительные команды для уровней "Хорошо" и "Отлично":

```
docsearch serve --config config.yml --addr :8080 # HTTP API
docsearch eval --config config.yml --dataset qa.jsonl #
```

Требования к ресурсам и вводу/выводу

1. При индексации документы читаются и обрабатываются последовательно/пакетами, без загрузки всего корпуса в память.
2. Повторная индексация не пересчитывает эмбединги для неизменённых документов (сравнение по хешу содержимого).
3. Обращения к внешним API выполняются с таймаутом и повторами при временных ошибках.
4. Секреты передаются только через переменные окружения и не логируются.
5. В режиме `llm.provider: mock` утилита работает без сети и даёт детерминированный результат — это нужно для тестов.

Алгоритмические требования

Нужно реализовать:

1. парсинг и нормализацию документов;
2. чанкование с перекрытием;
3. вычисление эмбедингов через выбранный провайдер;
4. векторное хранилище и поиск top-k по косинусной близости;
5. сборку промпта из найденного контекста (RAG);
6. вызов LLM и формирование ответа со ссылками на источники;
7. обработку случая «ответа нет в документации» по порогу `min_score`.

Критерии приёмки

Минимум:

1. чтение конфигурации из YAML и ключей из переменных окружения;
2. индексация корпуса md/txt с чанкованием;
3. вычисление эмбедингов через один провайдер;
4. векторный поиск top-k по косинусной близости;
5. RAG-ответ через LLM со ссылками на источники;
6. режим mock для LLM без обращения к сети;
7. unit-тесты чанкования и косинусной близости.

Хорошо:

1. инкрементальная индексация по хешу документов;
2. порог релевантности и корректный ответ «нет в документации»;
3. HTTP API (serve);
4. таймауты и повторы для внешних API;
5. поддержка формата html.

Отлично:

1. локальные эмбединги без внешнего API;
2. команда eval с метриками качества поиска (например, recall@k и доля корректных цитат) на размеченном наборе;
3. приближённый индекс (например, HNSW) и сравнение скорости/качества с полным перебором;
4. поддержка формата pdf.

Достаточно выбрать 1–2 пункта уровня «Отлично» и сделать их качественно, с тестами.

Общие требования к сдаче

Решение оформляется как самостоятельный проект на Go (модуль, go 1.20).

Обязательная структура проекта:

```
/cmd/docsearch      CLI
/internal/...       (corpus, chunk, embed, store, retrieve, llm, rag)
/configs            config.yml
/testdata           qa.jsonl
/docs
README.md
Makefile            test, build, demo
```

Обязательные команды:

```
make test
make build
make demo
```

В README.md должны быть описаны:

1. назначение проекта;
2. сборка и запуск;
3. формат конфигурации YAML и переменные окружения;
4. формат входных документов и выходного ответа;
5. примеры команд;
6. краткое описание RAG-пайплайна (чанкование, эмбединги, поиск, генерация);
7. известные ограничения;
8. результаты контрольного запуска.

Контрольная сдача и оценивание

В репозитории должен быть фиксированный набор /testdata/control: небольшой корпус документов, набор вопросов и ожидаемые источники. Команда make demo запускает решение в режиме mock (без обращения к внешним API) и сохраняет результат в предсказуемые файлы — это позволяет проверить пайплайн без ключей и сети.

Оценка решения:

Критерий	Вес
Корректность RAG-пайплайна и качество ответов с цитатами	40%
Тесты, включая mock и граничные случаи	20%
Работа с внешними API: ключи из env, таймауты, ошибки	15%
Качество конфигурации, README и воспроизводимость	15%
Простота архитектуры и читаемость кода	10%

В /docs/reshenie.md нужно на 1–2 страницы описать стратегию чанкования, выбор моделей эмбедингов и LLM, схему поиска, формат промпта, известные ограничения и идеи для улучшения.

Требования к исходному коду и истории коммитов

1. Код читаемый: понятные имена, разделение по пакетам, без «мёртвого» кода.
2. Ошибки возвращаются явно и с контекстом (какой документ/этап/запрос).
3. Никаких panic на штатных и некорректных входных данных.
4. Осмысленные атомарные коммиты, описывающие изменения логики; без сообщений вида fix, wip, final2.
5. gofmt/go vet без замечаний; проект собирается командой make build.

Справочные материалы

- [Retrieval-augmented generation](#) — генерация ответов с опорой на найденные документы.
- [Косинусное сходство](#) — мера близости векторов при поиске.
- [Векторное представление слов](#) — основа эмбедингов текста.
- [Большая языковая модель](#) — модель генерации ответа.